

1 Abstract

This report evaluates the integrity of High-Performance Liquid Chromatography (HPLC) chromatographic data, specifically addressing severe data quality challenges including missing metadata, negative volume values, and high signal variance. The primary objective is to determine whether robust normalization or dynamic baseline adjustment is required to mitigate process instability. Analysis reveals a critical discrepancy between the data analysis plan and the execution results: the initial data integrity check reported zero filtered rows, contradicting the existence of massive missingness (75%) in metadata columns and negative volume values identified in the Exploratory Data Analysis (EDA). Consequently, the reported Signal-to-Noise Ratio (SNR) values and subsequent conclusions regarding baseline correction are likely based on an unfiltered, full dataset containing invalid records. This report concludes that the current analysis phase has failed to isolate process instability or exclude physical anomalies, thereby undermining the validity of the SNR metrics and the proposed baseline correction strategies. Rigorous filtering and re-evaluation of the dataset are required before proceeding to normalization strategy evaluation and calibration curve construction.

2 Introduction

The dataset under investigation represents a multi-stage purification process characterized by significant data quality issues. Specifically, the UV signal columns exhibit high variance (standard deviation ≈ 390 mAU), which may mimic baseline shifts, while critical quantitative variables such as 'Conc_B_ml' are missing in over 65% of records. Furthermore, the metadata columns 'chromatography_stage' and 'Sample.Code' suffer from massive missingness (75%), and the 'volume_ml' variable contains negative values representing physical anomalies. The motivation for this data analysis workflow is to rigorously evaluate data integrity to determine if robust normalization (using '_min' flow variables) or dynamic baseline adjustment (per 'run_no') is required to mitigate process instability, rather than relying on static baseline correction. The primary goal is to assess if high UV signal variance stems from instrument noise or process drift, specifically accounting for the severe missingness in concentration and pH data. The analysis must define quantitative interpretation by constructing calibration curves using 'UV_1.280_mAU' peak areas against available concentration points to establish response factors for proxy quantification. Furthermore, the analysis must establish a strategy for handling physical anomalies and exclude unreliable metadata to ensure unbiased variance decomposition between process conditions and instrument noise.

3 Methodology

The data analysis plan was structured into three sequential steps designed to address data integrity, normalization strategies, and calibration curve construction.

Step 1: Data Integrity & Anomaly Filtering. The objective was to load the 'chromatography_combined.csv' dataset and filter rows based on specific criteria: 'chromatography_stage' set to 'Preparation' or 'Cleanup', 'Sample.Code' set to 'Blank' or 'Control', and 'volume_ml' ≥ 0 . The plan required calculating the Signal-to-Noise Ratio (SNR) for 'UV_1.280_mAU' and 'UV_2.260_mAU' using the standard deviation of the signal divided by the median absolute deviation (MAD) of the signal during the stable baseline period (the first 10% of 'volume_ml'), while avoiding low-variance '_min' flow variables. The expected output included a count of filtered rows, summary statistics for UV signals and conductivity, and average SNR per run number.

Step 2: Normalization Strategy Evaluation. This step aimed to implement two distinct strategies to address signal variance: 1) Static baseline correction (subtracting the mean of the run), and 2) Dynamic normalization using the mean of 'System_flow_ml_min' for the specific 'run_no' to scale UV signals. The methodology involved addressing missingness in 'pH_ml' by excluding rows with missing values prior to calculations. The comparison focused on the variance of normalized signals between methods.

Step 3: Calibration Curve Construction for Proxy Quantification. This final step involved filtering rows where 'Conc_B_ml' was missing and handling missingness in 'pH_ml' by dropping rows containing any NaN values. The 'UV_1.280_mAU' signal was aggregated to the 'run_no' level (mean signal) and fitted against 'Conc_B_%' using linear regression. The output was intended to provide the regression equation, R-squared value, standard error of estimate, and a statistical significance summary.

4 Results and Discussion

None

5 Conclusion

The current data analysis workflow has encountered a critical failure in the initial data integrity phase. While the analysis plan explicitly required filtering out rows with negative volumes and unreliable metadata (missing chromatography stage or sample code), the resulting report indicates that zero rows were filtered. This null result is critically inconsistent with the Exploratory Data Analysis (EDA) findings, which identified massive missingness (75%) in the specified metadata columns and the presence of negative volume values. This discrepancy suggests that the filtering logic either failed to execute correctly or the data source for the report does not reflect the actual dataset state. Consequently, the subsequent summary statistics and SNR calculations presented in the report are likely based on the unfiltered, full dataset, potentially including invalid records and physical anomalies. This undermines the validity of the reported SNR values and the subsequent conclusions regarding baseline correction. Until the data integrity issue is resolved and the dataset is properly cleaned to exclude unreliable metadata and physical anomalies, the analysis cannot proceed to the normalization strategy evaluation or calibration curve construction with confidence. Future work must prioritize debugging the filtering step and re-running the analysis on a cleaned dataset to ensure unbiased variance decomposition.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

def Call_Code_Updates():
    # Load the dataset
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Debug: Print actual column names
    print("Actual columns in dataframe:", list(df.columns))

    # Filter 1: chromatography_stage in ['Preparation', 'Cleanup']
    # Filter 2: Sample_Code in ['Blank', 'Control']
    # Filter 3: volume_ml < 0
    mask_stage = df['chromatography_stage'].isin(['Preparation', 'Cleanup'])
    mask_sample = df['Sample_Code'].isin(['Blank', 'Control'])
    mask_volume = df['volume_ml'] < 0

    filtered_df = df[mask_stage & mask_sample & mask_volume]

    # Calculate SNR per run_no using vectorized groupby operations
    # Definition: Std(signal) / MAD(signal) during first 10% of volume_ml per run
    columns_to_analyze = ['UV_1_280_mAU', 'UV_2_260_mAU']
    run_avg_snr = {}

    for col in columns_to_analyze:
        # Group by run_no and calculate baseline stats directly on grouped data
```

```

group_stats = filtered_df.groupby('run_no')[col].apply(
    lambda x: (np.std(x.iloc[:int(len(x)*0.10)], ddof=0),
               np.median(np.abs(x.iloc[:int(len(x)*0.10)] - np.median(x.
                               iloc[:int(len(x)*0.10)])))
               ) if len(x) > 0 else (0, 0)
)

# Calculate SNR for each run
stds = group_stats.index.map(lambda x: group_stats.loc[x][0])
mads = group_stats.index.map(lambda x: group_stats.loc[x][1])

# Avoid division by zero
snrs = np.where(mads == 0, 0.0, stds / mads)

# Store SNRs for this column
for run_id, snr in zip(group_stats.index, snrs):
    if run_id not in run_avg_snr:
        run_avg_snr[run_id] = []
    run_avg_snr[run_id].append(snr)

# Calculate final average SNR per run (average across columns)
final_snr_per_run = {}
for run_id, snr_list in run_avg_snr.items():
    final_snr_per_run[run_id] = np.mean(snr_list)

# Prepare Output 3: Text block with average SNR per run_no
snr_text = "Average_SNR_per_run_no:\n"
for run_id in sorted(final_snr_per_run.keys()):
    snr_text += f"{run_id}: {final_snr_per_run[run_id]:.4f}\n"

# Prepare Output 1: Count of rows filtered
count_filtered = len(filtered_df)

# Prepare Output 2: Summary stats
# Dynamically select columns that exist in the dataframe
existing_cols = [col for col in ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
                 if col in filtered_df.columns]

if not existing_cols:
    summary_stats = pd.DataFrame()
else:
    summary_stats = filtered_df[existing_cols].agg(['mean', 'std', 'min', 'max',
                                                  ''])
    # Align column names to match actual output
    summary_stats.columns = ['Mean', 'Std', 'Min']

# Ensure output directory exists
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# Create the markdown report
report_lines = []
report_lines.append("#_Data_Integrity_Report\n")

report_lines.append(f"##_1._Row_Filter_Count\n")
report_lines.append(f"Total_rows_filtered_by_metadata_and_volume:_{**{
    count_filtered}}*\n")

report_lines.append("##_2._Summary_Statistics\n")

```

```

report_lines.append("|_Variable_|_Mean_|_Std_|_Min_|_Max_|_\\n")
report_lines.append("|-----|-----|-----|-----|-----|\\n")

# Only iterate over columns that exist in summary_stats
for var in existing_cols:
    if var in summary_stats.columns:
        row = summary_stats[var].to_dict()
        report_lines.append(f"|_{var}_|_{row['Mean']:.4f}|_{row['Std']:.4f}|_
            _{row['Min']:.4f}|_{row['Max']:.4f}|_\\n")

report_lines.append("##_3._Average_SNR_per_run_no\\n")
report_lines.append(snr_text)

# Write to file
report_path = os.path.join(output_dir, 'data_integrity_report.md')
with open(report_path, 'w') as f:
    f.write('\\n'.join(report_lines))

print(f"Report saved to_{report_path}")

Call_Code_Updates()

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter for required variables and exclude rows with missing pH_ml
    required_vars = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'System_flow_ml_min',
        'Sample_flow_ml_min', 'pH_ml']
    df_filtered = df[required_vars].copy()
    df_filtered = df_filtered.dropna(subset=['pH_ml'])

    # Strategy 1: Static baseline correction (subtracting the mean of the run)
    df_static = df_filtered.copy()
    for var in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        df_static[var] = df_static[var] - df_static[var].mean()

    # Calculate variance (IQR) for static method
    df_static['variance_iqr'] = df_static[['UV_1_280_mAU', 'UV_2_260_mAU']].apply(
        lambda x: x.quantile(0.75) - x.quantile(0.25), axis=1
    )

    # Strategy 2: Dynamic normalization using the mean of System_flow_ml_min for
    # the specific run_no
    df_dynamic = df_filtered.copy()
    # Group by run_no and calculate mean of System_flow_ml_min
    flow_means = df_dynamic.groupby('run_no')['System_flow_ml_min'].transform('
        mean')

    for var in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        df_dynamic[var] = df_dynamic[var] / flow_means

```

```

# Calculate variance (IQR) for dynamic method
df_dynamic['variance_iqr'] = df_dynamic[['UV_1_280_mAU', 'UV_2_260_mAU']].
    apply(
        lambda x: x.quantile(0.75) - x.quantile(0.25), axis=1
    )

# Prepare data for boxplot
boxplot_data = pd.DataFrame({
    'run_no': df_static['run_no'],
    'variance_iqr': df_static['variance_iqr'],
    'method': 'Static'
})
boxplot_data = pd.concat([
    boxplot_data,
    pd.DataFrame({
        'run_no': df_dynamic['run_no'],
        'variance_iqr': df_dynamic['variance_iqr'],
        'method': 'Dynamic'
    })
], ignore_index=True)

# Find run_no with highest variance (using Static method as primary, or max of
    both if needed, plan implies "highest variance" generally)
# Let's find the run_no with the highest average variance across both methods
    to be robust, or just max of the combined set.
# Re-reading plan: "run_no with highest variance". Let's take the max of the
    combined variance_iqr column.
max_var_row = boxplot_data.loc[boxplot_data['variance_iqr'].idxmax()]
target_run_no = max_var_row['run_no']

# Subset for line plot: 1000 rows if > 1M total rows in original filtered data
total_rows = len(df_filtered)
subset_size = min(1000, total_rows)
subset_df = df_filtered.sample(n=subset_size, random_state=42)

# Line plot: Raw vs Normalized UV_1_280_mAU for target_run_no
# We need to align the subset for static and dynamic to the same run_no
# FIX: Derive subsets from the pre-processed dataframes (df_static and
    df_dynamic) instead of the filtered copy
subset_static = df_static[df_static['run_no'] == target_run_no].copy()
subset_dynamic = df_dynamic[df_dynamic['run_no'] == target_run_no].copy()

# Ensure we have enough data points for a meaningful line plot, otherwise take
    all
if len(subset_static) < 10:
    # This block is now redundant as we already loaded from df_static/dynamic,
        but kept for safety if data is empty
    pass

# Plotting
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Subplot 1: Boxplot
axes[0].boxplot(boxplot_data.groupby('method')['variance_iqr'], labels=['
    Static', 'Dynamic'], patch_artist=True)
axes[0].set_title('Variance_IQR_Comparison_by_Normalization_Strategy')
axes[0].set_ylabel('IQR')
axes[0].legend(loc='upper_right')
axes[0].grid(axis='y', alpha=0.3)

```

```

# Subplot 2: Line plot
if len(subset_static) > 0:
    # FIX: Plot raw data from df_static (mean subtracted) and df_dynamic (
    #       normalized)
    # The 'Raw' line should show the static-corrected data (which is what
    #       df_static contains after subtraction)
    # The 'Static Normalized' label in the original code was misleading; we
    #       will plot the static-corrected data as 'Raw' relative to the static
    #       method's baseline
    # The 'Dynamic Normalized' line will show the dynamic-corrected data
    axes[1].plot(subset_static.index, subset_static['UV_1_280_mAU'], label='
    Raw', alpha=0.7)
    axes[1].plot(subset_dynamic.index, subset_dynamic['UV_1_280_mAU'], label='
    Dynamic Normalized', alpha=0.7)
    axes[1].set_title(f'UV_1_280_mAU_for_run_no_{target_run_no}')
    axes[1].set_ylabel('mAU')
    axes[1].legend()
    axes[1].grid(alpha=0.3)
else:
    axes[1].text(0.5, 0.5, 'No data found for target_run_no', transform=axes
    [1].transAxes, ha='center')
    axes[1].set_title(f'UV_1_280_mAU_for_run_no_{target_run_no}')

plt.tight_layout()
plt.savefig('/workspace/normalization_strategy_evaluation.png', dpi=300,
            bbox_inches='tight')
plt.close()
###

```

Listing 2: Code.2